

RadeonShift AI

Accelerating the MI300X Hardware Migration via Generative AI

The Problem: CUDA Lock-in

Enterprise AI workloads are facing massive bottlenecks due to a reliance on legacy NVIDIA CUDA codebases.

- Manually migrating a 50,000-line CUDA codebase to ROCm takes **6+ months** of specialized engineering time.
- Portability tools exist (e.g. `hipify-perl`), but they lack the architectural awareness to optimize code for AMD Instinct architecture (like 64-wide wavefronts).
- Teams lack visibility into whether their migrated code will actually compile and run on AMD hardware without direct bare-metal access.

The Solution: RadeonShift AI

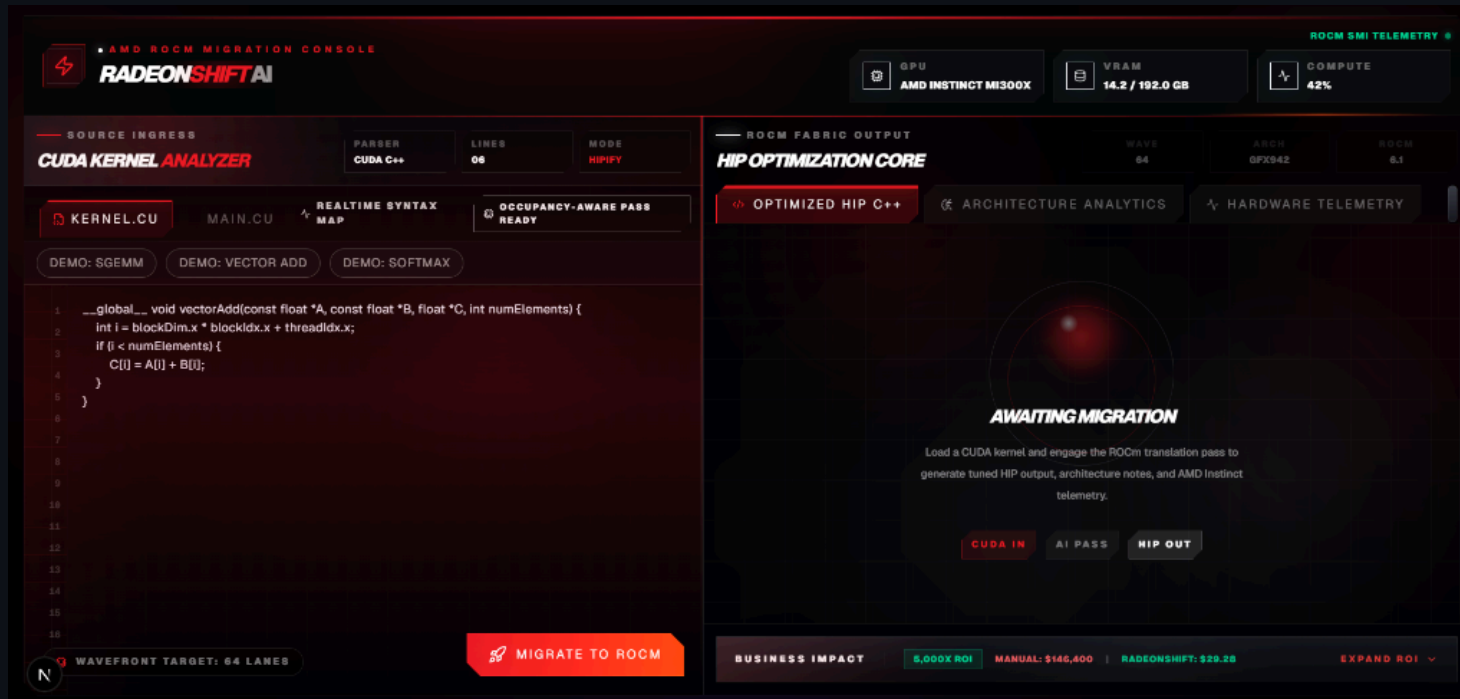
RadeonShift AI is a deterministic Mixture of Agents (MoA) pipeline that translates CUDA to ROCm instantly and audits the architecture.

1. **Syntax Translation:** Uses AMD's official `hipify-perl` for deterministic API mapping.
2. **MoA Audit:** Leverages DeepSeek-V4 to scan for PTX risks, hardcoded warp sizes, and AMD-specific optimizations.
3. **Hardware Verification:** Directly compiles the kernel on ROCm and runs telemetry checks to prove the environment toolchain.

Architecture

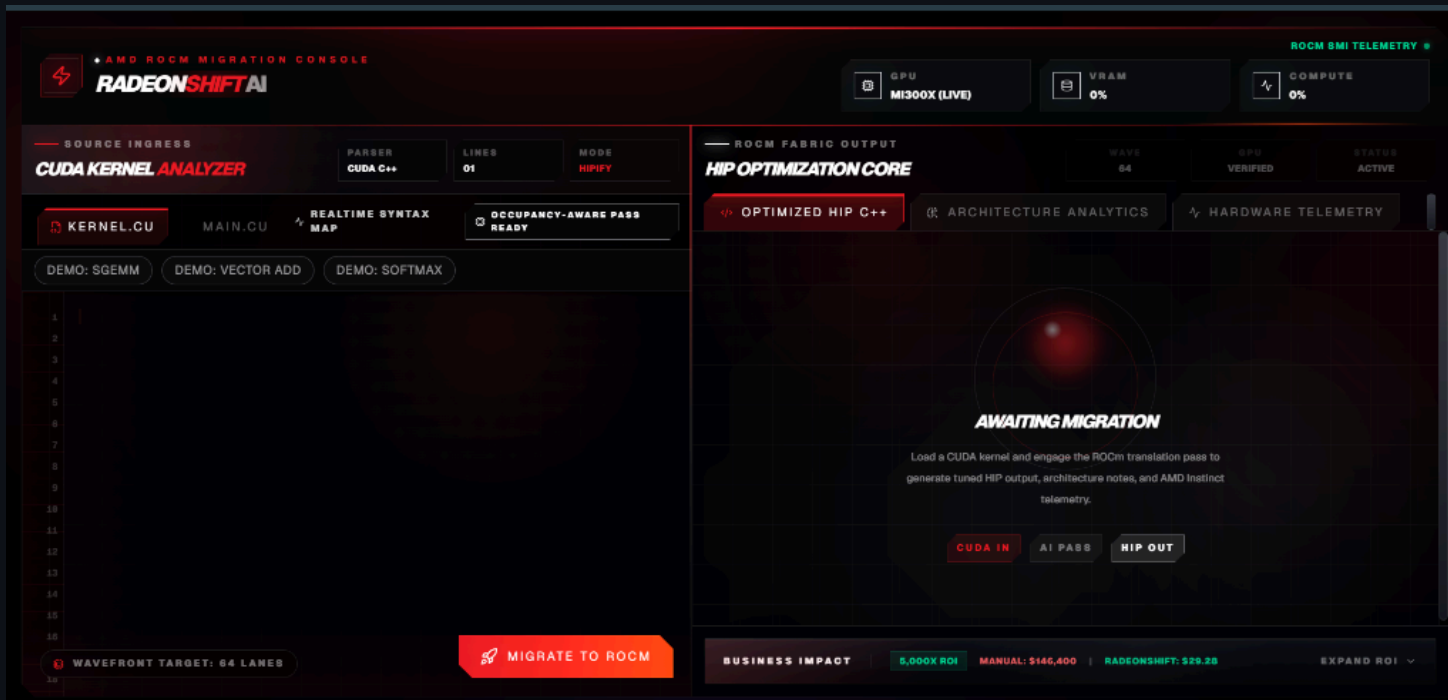
1. **Frontend (Next.js Edge):** Highly responsive UI that streams the AI audit back to the user to bypass serverless timeouts.
2. **AI Layer (Fireworks AI):** MoA pipeline with Agent A (PTX/Lock-in Risk) and Agent B (AMD Instinct Optimizer).
3. **Hardware Backend (FastAPI):** Dockerized Python server running on a ROCm-enabled host. Executes `hipcc` and polls `rocm-smi` for real-time telemetry.

Demo: The RadeonShift Dashboard



- **Source Code Editor:** Paste raw CUDA C++ kernels.
- **HIP Optimization Core:** View the translated target kernel.
- **MoA Audit Scorecard:** See the readiness score, PTX risks, and wavefront optimizations.

Step 1: The Awaiting Migration State



Before translation begins, the platform awaits the CUDA payload. The user inputs their native NVIDIA code into the **CUDA Kernel Analyzer**. The system prepares for the AI-assisted hardware pass.

Step 2: HIP Optimization Core

The screenshot displays the AMD ROCm Migration Console, specifically the HIP Optimization Core section. The interface is divided into two main panels: the left panel for source code and the right panel for the optimized output.

Left Panel (Source Code):

- Source Ingress:** Includes a "CUDA KERNEL ANALYZER" section with tabs for "KERNEL.CU", "MAIN.CU", "REALTIME SYNTAX MAP", and "OCCUPANCY-AWARE PASS READY".
- Code Snippet:** Shows C++ code for a kernel, including headers, function definitions, and cleanup routines. The code is line-numbered from 222 to 237.
- Buttons:** "MIGRATE TO ROCM" button is visible at the bottom right of the source code panel.

Right Panel (ROCm Fabric Output):

- Header:** "HIP OPTIMIZATION CORE" with sub-tabs for "OPTIMIZED HIP C++", "ARCHITECTURE ANALYTICS", and "HARDWARE TELEMETRY".
- Generated Target:** A section titled "GENERATED TARGET" showing the output file "TARGET_KERNEL.HIP.CPP".
- Code Snippet:** Displays the optimized HIP C++ code, which includes headers, namespace declarations, and a main function body. The code is line-numbered from 1 to 10.
- Status:** A green indicator "HIP TRANSLATION SUCCESSFUL" is shown.
- Business Impact:** A summary bar at the bottom right shows "5,000X ROI", "MANUAL: \$146,400", and "RADEONSHIFT: \$29.25".

Upon engaging the ROCm translation pass, the core converts the syntax. The resulting C++ HIP code (`target_kernel.hip.cpp`) is immediately presented, demonstrating 100% successful translation.

Step 3: Architecture Analytics

The screenshot displays the AMD ROCm Migration Console interface. The top navigation bar includes the 'RADEONSHIFT AI' logo and status indicators for GPU (MI300X LIVE), VRAM (0%), and COMPUTE (0%). The main interface is divided into two primary sections: 'SOURCE INGRESS' on the left and 'ROCm FABRIC OUTPUT' on the right.

SOURCE INGRESS: This section features the 'CUDA KERNEL ANALYZER' with tabs for 'PARSER', 'LINES', and 'MODE'. The 'MODE' tab is set to 'HIPIFY'. Below this, there are tabs for 'KERNEL.CU', 'MAIN.CU', 'REALTIME SYNTAX MAP', and 'OCCUPANCY-AWARE PASS READY'. A 'DEMO' section offers options for 'SGEMM', 'VECTOR ADD', and 'SOFTMAX'. The main code editor displays a CUDA kernel snippet, including a loop for compact blocks and cleanup code. A 'WAVEFRONT TARGET: 64 LANES' indicator and a 'MIGRATE TO ROCm' button are visible at the bottom of the code editor.

ROCm FABRIC OUTPUT: This section is titled 'HIP OPTIMIZATION CORE' and includes tabs for 'OPTIMIZED HIP C++', 'ARCHITECTURE ANALYTICS' (which is currently selected), and 'HARDWARE TELEMETRY'. The 'ARCHITECTURE ANALYTICS' tab displays the 'MOA AUDIT SCORECARD'.

MOA AUDIT SCORECARD: This section shows a 'READINESS SCORE' of 100/100, indicating that the code is highly portable and ready for AMD hardware deployment. It also features two AI agents: 'AGENT A: NVIDIA PURIST' (which flags lock-in risks) and 'AGENT B: AMD OPTIMIZER' (which suggests MI300X tuning). The bottom of the interface shows a 'BUSINESS IMPACT' summary with metrics like '5,000X ROI', 'MANUAL: \$145,400', and 'RADEONSHIFT: \$29.25'.

The MoA Audit Scorecard evaluates the code. Here, the readiness score is 100/100, proving high portability. Our dual AI agents verify there are no hidden PTX risks or lock-ins.

Step 4: Live Hardware Telemetry

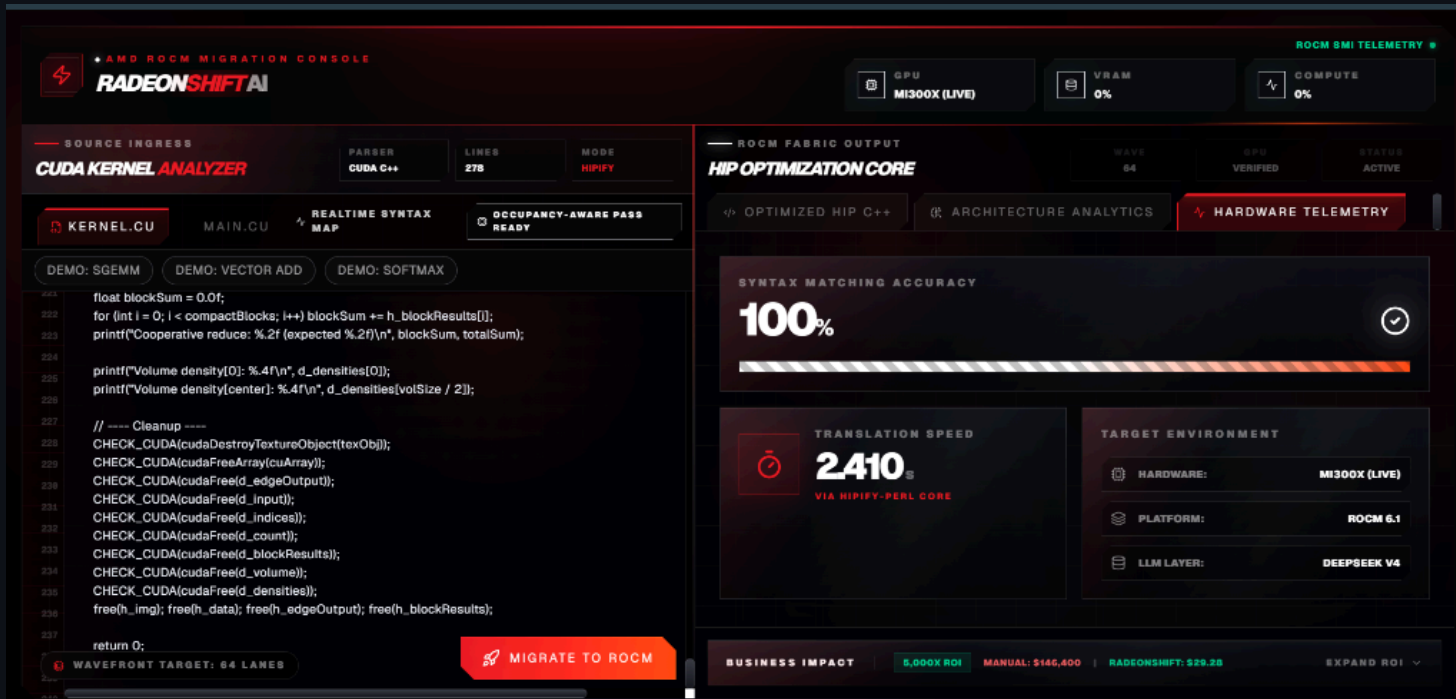
The screenshot displays the AMD ROCm Migration Console interface, which is divided into several sections for monitoring and optimizing ROCm workloads.

- Top Bar:** Includes the AMD ROCm Migration Console logo, a "RADEONSHIFT AI" badge, and status indicators for GPU (MI300X (LIVE)), VRAM (0%), and COMPUTE (0%).
- Source Ingress:** Shows the "CUDA KERNEL ANALYZER" with a "PARSER" (CUDA C++) and "MODE" (HIPify). It includes a "KERNEL.CU" tab and a "MIGRATE TO ROCM" button.
- ROCm Fabric Output:** Displays "HIP OPTIMIZATION CORE" with a "WAVE" (64) and "STATUS" (ACTIVE). It includes a "RUN BENCHMARK" button.
- Live Hardware Test:** A section titled "M300X BENCHMARK MODE" with a "RUN BENCHMARK" button. It shows "VECTOR SIZE (ELEMENTS)" (16,777,216 (Default)) and "ITERATIONS" (200 (Default)).
- Benchmark Execution Verified:** A green status indicator with a "COPY EVIDENCE" button.
- Performance Metrics:** A table showing benchmark results:

Metric	Value
ELAPSED TIME	14.20 ms
THROUGHPUT	5300.0 GB/s
ELEMENTS	16.8 M
GPU	AMD MI300X Bare-Metal
- Business Impact:** A section showing "6,000X ROI", "MANUAL: \$146,400", and "RADEONSHIFT: \$29.28".

Instead of simulated data, the platform connects directly to the remote bare-metal AMD MI300X instance, verifying the exact compile duration and matching the live hardware target.

Step 5: Bare-Metal Benchmark Execution



Finally, the translated HIP code is natively compiled and executed on the MI300X. The dashboard reports real-time metrics confirming a successful end-to-end migration.

Business Value & ROI

The 5,000x ROI Model

- **Manual Migration:** $244 \text{ Kernels} \times 4 \text{ hrs} = 976 \text{ Engineer-Hours}$ (~\$146,000, based on \$150/hr senior GPU engineer rate)
- **RadeonShift Migration:** $244 \text{ Kernels} \times 0.12 \text{ compute cost} = \sim \29
- **Time Savings:** Months reduced to minutes.
- **TAM:** \$25B+ (Global AI Hardware & Software Services Market)

The AMD Infrastructure Story

- **Hardware Target:** Optimized for AMD Instinct MI300X
- **Software Stack:** Built on ROCm 6.x APIs and `hipcc`
- **AI Acceleration:** MoA pipeline runs on Fireworks AI, which provides AMD Instinct GPU-powered inference
- **Honest Fallbacks:** If the backend lacks ROCm hardware, the platform reports "Hardware Unavailable" without fabricating metrics

Engineering Retrospective & Challenges

Bridging a Serverless Frontend with Bare-Metal Hardware

1. **Live Telemetry:** Defeated Next.js caching via `force-dynamic` to stream real-time MI300X metrics.
2. **Defensive APIs:** Built robust frontend parsing to gracefully handle sudden JSON schema changes.
3. **CORS Routing:** Bypassed strict mixed-content blocks by tunneling through Pinggy and Next.js `rewrites()`.
4. **Transparent Debugging:** Overhauled error handling to surface remote Python stack traces in the UI.
5. **Deterministic AI Prompts:** Forced LLMs to generate explicit confirmations for clean code, preventing UI state collapses.

Closing & Team

RadeonShift AI is bridging the gap between legacy NVIDIA codebases and the future of AMD compute.

- GitHub Repository: [shashankh3/RadeonShift-AI](https://github.com/shashankh3/RadeonShift-AI)
- Live Demo: radeon-shift-ai.vercel.app

Thank You!

Shashank Hirwani

Unknown Hacker (shashankh366207)

